

# תרגיל בית 3 ~ אלגוריתמים

שחר פרץ

12 ביוני 2026

..... (1) .....

יהי גרף מכוון וקשיר  $G = (V, E)$ , פונקציית משקל  $w: E \rightarrow \mathbb{R}$  וקשת  $e \in E$ .

הערה לשאלה זו: האלגוריתם הגנרי יכול ליצור כל ע"מ, כי קרוסקל יכול ליצור כל ע"מ (הוכחנו בתרגול).

(א) נמצא אלגוריתם הסיבוכיות לינארית הבודק האם קיים ע"מ המכיל את  $e$ .

• **אלגוריתם:** נסמן  $r := w(e)$  ו- $e = (u, v)$ . נבנה את  $G_{<r}$ , תת-הגרף של כל הקשתות של  $p \in E, w(p) < r$  (גם את  $e$  נסיר). נבדוק האם ב- $G_{<r}$  באמצעות BFS האם יש מסלול  $u \rightsquigarrow v$ . נחזיר תשובה שלילית במידה וישנו מסלול כזה.

• **נכונות:** נוכיח שאין מסלול הכולל את  $u, v$  ב- $G_{<r}$  אם"מ  $e$  על ע"מ כלשהו.

מנכונות האלגוריתם הגנרי, קשת נמצאת על ע"מ כלשהו אם"מ ישנה סיטואציה בה היא אינה מוסרת לפי הכלל האדום.

$\Rightarrow$  אם  $e$  על ע"מ כלשהו, אז היא לא הוסרה ע"פ הכלל האדום, כלומר היא איננה הקשת הכבדה ביותר באף מעגל. אם בשלילה ישנו מסלול  $u \rightsquigarrow v$  ב- $G_{<r}$ , אז מצאנו מעגל  $u \rightsquigarrow v \rightarrow u$  כך שכל קשתותיו במשקל קטן ממש מ- $w(e)$  (פרט ל- $e$ ), וזו סתירה.

$\Leftarrow$  אם אין מסלול  $u \rightsquigarrow v$  ב- $G_{<r}$ , ובשלילה היא לא באף ע"מ, אז בהכרח היא מוסרת לפי הכלל האדום, כלומר קיים מעגל  $u \rightsquigarrow v \rightarrow u$  שבו  $e = (u, v)$  (קשת הכבדה ביותר (כבדה ממש, אחרת נוכל להסיר לפי הכלל האדום קשת אחרת במקום, וזו סתירה כי היא איננה באף ע"מ) כלומר  $u \rightsquigarrow v$  מורכב מקשתות ממשקל קטן ממש מ- $w(u, v)$ , דהיינו מצאנו מסלול  $u \rightsquigarrow v$  ב- $G_{<r}$  וזו סתירה.

• **סיבוכיות:** עלות בניית  $G_{<r}$  היא עלות המעבר על הקשתות, שהיא  $|E|$  פעולות קבועות. עלות הרצת ה-BFS תהיה במקרה הגרוע  $\mathcal{O}(|E| + |V|)$ , וסה"כ נקבל  $\mathcal{O}(|E| + |V|)$ .

(ב) נמצא אלגוריתם בסיבוכיות  $\mathcal{O}(|V| + |E|)$  הבודק האם  $e$  נמצאת על כל ע"מ.

• **אלגוריתם:** נסמן  $r := w(e)$  ו- $e = (u, v)$ . נבנה את  $G_{\leq r}$ , תת-הגרף של כל הקשתות של  $p \in E, w(p) \leq r$  (בפרט את  $e$  לא נסיר). נבדוק האם ב- $G_{\leq r}$  באמצעות DFS האם יש מעגל הכולל את  $(u, v)$ . נחזיר תשובה שלילית במידה וישנו מעגל כזה.

• **נכונות:** נוכיח ש- $e$  על כל ע"מ אם"מ אין מעגל הכולל את  $(u, v)$  ב- $G_{\leq r}$ .

מנכונות האלגוריתם הגנרי, הקשת  $e$  תוסר מע"מ אם"מ הכלל האדום מתקיים. נבחין גם שלא אלגוריתם הגנרי יש בחירה איזו קשת הוא מסיר, במידה ויש מספר קשתות כבדות ביותר על אותו המעגל.

$\Rightarrow$  אם אין מעגל הכולל את  $u, v$  ב- $G_{\leq r}$ , ובשלילה היא לא נמצאת על ע"מ ספציפי, אז במהלך ריצת האלגוריתם הגנרי שיצר את הע"מ הזה נמצא מעגל  $u \rightsquigarrow v \rightarrow u$  כך ש- $e$  קשת כבדה ביותר. מכאן ש- $u \rightsquigarrow v$  מורכב מקשתות ממשקל לכל היותר  $r = w(e)$  (ייתכן שבמשקל שווה, כי לאלגוריתם יש חופש בחירה איזו קשת הוא מסיר) ובגלל ש- $e = (u, v)$  ב- $G_{\leq r}$ , מצאנו מעגל ב- $G_{\leq r}$  וזו סתירה.

$\Leftarrow$  אם  $e$  על כל ע"מ, אז היא מעולם לא מוסרת ע"פ הכלל האדום, ולכן בכל מעגל שנמצא הכולל את  $(u, v)$  היא איננה קשת כבדה ביותר. לכן בכל מעגל הכולל את  $(u, v)$  ישנה קשת ממשקל גדול ממש מ- $w(e)$ , והקונטראפוזיטיב של כך הוא שאין אף מעגל הכולל את  $(u, v)$  ב- $G_{\leq r}$ , כנדרש.

• **סיבוכיות:** עלות בניית הגרף היא  $\mathcal{O}(|E|)$  (מעבר על הקשתות), ועלות הרצת ה-DFS היא  $\mathcal{O}(|V| + |E|)$ . סה"כ  $\mathcal{O}(|V| + |E|)$ .

המשך בעמוד הבא

## (2)

יהי גרף  $G = (V, E)$  קשיר ולא מכוון, ממושקל עם  $w: E \rightarrow \mathbb{R}$ , ועפ"מ  $T$  של  $G$ . נגדיר  $G'$  ע"י הוספת  $v$  ל- $V$  וקשתות היוצאות ממנו לחלק מצמתי  $G$ . נמצא עפ"מ ב- $G'$  בזמן  $\mathcal{O}(|V| \log |V|)$ .

• **אלגוריתם:** נסמן ב- $T_E$  את הקשתות בעץ  $T$ , ונסמן ב- $G' = (V', E')$  את הגרף שאליו נוספו  $v$  והקשתות היוצאות ממנו (הן  $E' \setminus E$ ). עוד נסמן  $H = T_E \uplus (E' \setminus E)$ , ונריץ קרוסקל על תת-הגרף  $(V', H)$  של  $G'$ . נחזיר את התוצאה. (האיחוד אכן זר כי  $T_E$  תת-גרף של  $G$  ולא של  $G'$ ).

• **נכונות:** נוכיח שעץ  $T'$  ב- $(V', H) := G_H$  הוא עפ"מ ב- $G'$  אמ"מ הוא עפ"מ ב- $(V', H)$ .

- אם  $T'$  עפ"מ ב- $G'$  וגם הוא תת-גרף של  $G_H$ , אז בבירור הוא גם עפ"מ ב- $G_H$  (כי אחרת ישנו תת-עץ  $T''$  של  $G_H$  ממשקל  $w(T'') < w(T')$  ולכן זהו עץ ממשקל נמוך יותר גם ב- $G'$  כלומר  $T'$  אינו עפ"מ ב- $G'$  וזו סתירה).

- עתה יהי  $T'$  עפ"מ ב- $G_H$ , ונוכיח שהוא עפ"מ ב- $G'$ . לשם כך, נוכיח שישנו עפ"מ  $T_{\min}$  כלשהו ב- $G_H$  שהוא גם עפ"מ ב- $G'$ , ואז באופן מיידי מהכיוון הקודם  $w(T') = w(T_{\min})$  (כי  $T_{\min}$  יהיה עפ"מ ב- $G'$  ולכן עפ"מ ב- $G_H$ ).

נסמן ב- $S$  את החתך של  $v$  ושל כל בניו (כלומר כל מי ש- $T_E$  מחבר אליו). נבחין שבגלל שב- $V' \setminus S$  כל הקשתות המחברות קודקודים בתוך החתך הן קשתות ששייכות ל- $G$  המקורי, ישנו נוכל לבנות עפ"מ ביניים שיתחיל בחתך  $V' \setminus S$  ולהרחיב להרחיב אותו לעפ"מ יותר גדול כמו והיינו מריצים את האלגוריתם של פריס (למעשה אנו משתמשים כאן בנכונותו כשאנחנו מתירים לעצמנו להתחיל מעפ"מ של חתך ולטעון שיש הרחבה שלו לעפ"מ של כל העץ): ניקח קשתות קלות ביותר בין  $V' \setminus S$  ל- $S$  ונרחיב את החתך של  $V' \setminus S$  בהתאם, כמו באלגוריתם של פריס. נבחין שכל הקשתות שנצטרך להוסיף מהעפ"מ של  $V' \setminus S$  הן קשתות מ- $T_E \uplus (E' \setminus E)$ ; אחרת, ישנה קשת קלה ביותר מחוץ לקבוצה זו הפוגשת את אחד הקודקודים ב- $\{v\} \cup S$ , שהיא קלה יותר מכל שאר הקשתות במצב הנוכחי של החתך  $S$  לאחר שנוספו לו קודקודים. אם זו קשת ב- $E' \setminus E$  סיימנו, אחרת יכולנו לבחור קשת מ- $T_E$  משום שאחרת זו סתירה לנכונות הכלל הכחול באלגוריתם הגנרי ולכך ש- $T_E$  עפ"מ. סה"כ בנינו  $T_{\min}$  עפ"מ ב- $G_H$  וב- $G'$ . מנימוקים המצוינים לעיל  $T'$  עפ"מ ב- $G'$  כדרוש.

• **סיבוכיות:** נבחין ש- $|H| = |T|_E + |E' \setminus E|$ . מהגדרה  $|E' \setminus E| = k$  וכן מנוסחה  $|T_E| = |V| - 1$  סה"כ  $|H| = k + |V| - 1$ . בבירור  $|V'| = |V| + 1$ . לכן הרצת קרוסקל על  $G_H$  תארוך  $\mathcal{O}((k + |V|) \log |V|) = \mathcal{O}((k + |V| - 1) \log(|V| + 1))$ .

המשך בעמוד הבא

### (3)

נתון גרף מכוון ולא קשיר  $G = (V, E)$ , פונקציית משקל  $w: E \rightarrow \mathbb{R}$  חח"ע, ו- $k = \mathcal{O}(\log |V|)$  ו- $|E| = |V| + k$ . נמצא עפ"מ בגרף.

• **אלגוריתם:** נריץ DFS על הגרף. מכיוון שהגרף לא מכוון וקשיר, נקבל עץ.

לכל קודקוד שנוגע בקשתות אחוריות או חוצות, נקרא קודקוד אדומים. נעבור על הקודקודים האדומים לפי סדר גילוי עולה, נאמר  $v_1 \dots v_m$ . ראשית, כדי למצוא את הקשת הכבדה ביותר  $v_1 \rightarrow v_2$ , נטייל בסדר גילוי עולה עד ל- $v_2$  ונשמור בעבור  $(v_1, v_2)$  את הקשת הכבדה ביותר בדרך (עם הפנייה אליה). כנ"ל בעבור כל  $v_i, v_{i+1}$ . לאחר מכן, נחשב את הקשת הכבדה ביותר על הגרף בין  $(v_i, v_{i+2})$  ע"י המקסימום מבין  $(v_i, v_{i+1})$  ו- $(v_{i+1}, v_{i+2})$ , וכו'. נמשיך באופן דומה בעבור  $(v_i, v_{i+\ell})$  לכל  $\ell \in \{1 \dots k-1\}$ . נסמן את המשקל הקשת המקסימלית במסלול בעץ ששמרנו לכל זוג קודקודים כזה ב- $w_T(u, v)$ .

לבסוף, נבצע את השינוי הבא לעץ ה-DFS: לכל קשת שאינה קשת עץ  $e = (u, v)$ , אם  $w(e) < w_T(u, v)$  נחליף את הקשת הכבדה המשויכת ל- $w_T(u, v)$  בקשת האחורית  $(u, v)$ . אחרת, לא נשנה את הגרף. נחזיר את העץ שקיבלנו.

• **נכונות:** אנחנו מחזירים עץ, כי כאשר אנחנו מחליפים קשת שנמצאת על מסלול בעץ בקשת אחורית, אין אנו סוגרים מעגל, ועדיין מתקיים  $|E|_T = |V| - 1$  (כאשר  $E_T$  הקשתות בעץ), ולכן לא יצרנו רכיבי קשירות חדשים ואנו עדיין בעץ (למעשה, לאורך כל השלב האחרון נשמרת השמורה לפיה אנו מחזיקים עץ בזכרון). זהו עפ"מ כי נבחין שקשת נשללה מהעץ אמ"מ היא נמצאת על מעגל שבו היא הכבדה ביותר, כלומר, שלילת קשתות מעץ מתבצעת לפי הכלל האדום בלבד, ולכן אפשר להפעיל את נכונות האלגוריתם הגנרי.

• **סיבוכיות:** הליך הרצת ה-DFS יארוך  $\mathcal{O}(|E| + |V|)$ . נבחין שבגלל שיש  $|V| - 1$  קשתות בעץ, יש  $k + 1 = \mathcal{O}(k)$  קשתות מחוץ לעץ, ולכן  $2k + 2$  קודקודים אדומים. לכן הלולאה באמצע של חישוב  $w_T(u, v)$  לכל  $u, v$  אדומים, תארוך  $\mathcal{O}(k^2)$  פעמים (כי יש  $\binom{k}{2}$  זוגות של  $u, v$  אדומים). השלב הראשון של הלולאה עובר עליהם לפי סדר גילוי, ולכן נעבור על כל  $|E|$  פעם אחת בלבד. השלב השני אורך זמן קבוע. סה"כ עלות הלולאה הזו היא  $\mathcal{O}(|E| + k^2)$ . לבסוף הלולאה האחרונה תעבור על כל  $k + 1$  הקשתות שאינן קשתות עץ, ותבצע חישוב והשמה בזמן קבוע. סה"כ (ניעזר בכך ש- $|E| = |V| + k$ ):

$$\mathcal{O}((|E| + |V|) + (k^2 + |E|) + (k)) = \mathcal{O}(3|V| + 3k + k^2) = \mathcal{O}(|V| + k^2)$$

מכיוון ש- $k^2 = \mathcal{O}(\log^2 |V|)$  שהוא זניח ביחס לעלות לינארית ב- $|V|$ , הביטוי לעיל שקול אסימפטוטית ל- $\mathcal{O}(|V|)$ .

הערה לגודל: לא ברור לי למה ביקשתם סיבוכיות תלויה ב- $k$ , ומאידך נתתם חסם אסימפטוטי על  $k$  כתלות ב- $|V|$ . בכל מקרה סיפקתי ניתוח סיבוכיות כתלות ב- $k$  גם ללא הנתון  $k = \mathcal{O}(\log V)$ .

המשך בעמוד הבא

# (4)

נתון גרף  $G = (V, E)$  ופונקציית משקל  $w: E \rightarrow \mathbb{R}$ . תהי  $f: \mathbb{R} \rightarrow \mathbb{R}$ .

(א) נוכיח שאם  $f$  מונוטונית עולה חלש ו- $T$  עפ"מ ביחס לפונ' משקל  $w$ , אז  $T$  עפ"מ ביחס ל- $w \circ f$ .

הוכחה. בהרצאה/תרגול עם עמית הראנו שקרוסקל יכול להחזיר כל עץ פורש מינימלי, ובפרט את  $T$ . לכן קיים סידור  $e_1 \dots e_m$  כך ש- $w(e_1) \leq w(e_2) \leq \dots \leq w(e_m)$  ושריצת קרוסקל על הצמתים לפי הסדר הזה מחזירה את  $T$ . משום ש- $f$  מונוטונית עולה חלש, אם  $w(e_i) \leq w(e_j)$  אז  $f(w(e_i)) \leq f(w(e_j))$ . לכן גם  $(f \circ w)(e_1) \leq \dots \leq (f \circ w)(e_m)$ . האלגוריתם של קרוסקל עובר על הקשתות לפי סדרן משקלן (ואינו תלוי במשקל לאחר הסידור), ולכן גם ביחס ל- $f \circ w$  ישנו סידור בעבורו הוא יחזיר את  $T$ . ■

(ב) אם  $f$  מונוטונית עולה חזק, אז לכל  $T$  מתקיים ש- $T$  עפ"מ ביחס ל- $w$  אמ"מ  $T$  עפ"מ ביחס ל- $w \circ f$ .

הוכחה. אם  $T$  עפ"מ ביחס ל- $w$ , אז מכיוון ש- $f$  מונוטונית עולה חזק ובפרט עולה חלש, מהסעיף הקודם הוא גם עפ"מ ביחס ל- $w \circ f$  וסיימנו.

אם  $T$  עפ"מ ביחס ל- $w \circ f$ , אז בדומה לסעיף קודם יש ריצת קרוסקל עם  $(f \circ w)(e_1) \leq \dots \leq (f \circ w)(e_m)$  סידור של הקשתות. מכיוון ש- $f$  עולה חזק מובטח לנו שהא"שים נשמרים גם בעבור  $w(e_1) \leq \dots \leq w(e_m)$ , כלומר ריצת קרוסקל בסדר  $e_1 \dots e_m$  תחזיר עפ"מ תקין גם ביחס ל- $w$ , כנדרש. ■

(ג) נניח ש- $|E| = k|V|$  ו- $k = 2026$  קבוע. עוד נניח שלכל  $e \in E$  מתקיים  $w(e) \in [0, |V|]$ . נתונה קבוצה  $A \subset [0, |V|]$  מגודל  $|A| = \mathcal{O}(1)$  קבוע. נמצא אלגוריתם יעיל הבודק האם קיים עפ"מ שמשקלי כל קשתותיו שייכים ל- $A$ .

• **אלגוריתם:** נגדיר את הפונקציה:

$$f(x) = \begin{cases} 2x & x \in A \\ 2\lfloor x \rfloor + 1 & x \notin A \end{cases}$$

ראשית נרכיב את  $f$  על  $w$  לקבלת משקלי קשתות חדשים בגרף. עתה נריץ קרוסקל (עם Union Find) על הגרף, עם שינוי להליך המיון: ננצל את העובדה שלאחר הרכבה ב- $f$ , בהכרח המשקלים מספרים שלמים בין  $1$  ל- $|V| + 1$ , או שהם כפולות של  $2$  של ערכי  $A$ . סה"כ, ישנה כמות סופית של אפשרויות למשקלים (ליתר דיוק,  $|V| + |A|$  אפשרויות) ולכן ניתן להפעיל את count sort שלמדנו במבנ"ת. לבסוף, ניקח את העץ שקרוסקל החזיר ונוודא שקשתותיו אכן ב- $A$ . אם זה המצב, נחזיר תשובה חיובית. אחרת, נחזיר תשובה שלילית.

• **נכונות:** נוכיח שקרוסקל מחזיר עץ שקשתותיו ב- $A$  ביחס ל- $w \circ f$  אמ"מ ישנו כזה ביחס ל- $w$ .

⇐ אם מצאנו עץ ביחס ל- $w \circ f$  שכל קשתותיו ב- $A$ , אז הצמצום  $f|_A$  פונקציה מונוטונית עולה חזק (פשוט  $2x$ ) ולכן במקרה זה נוכל להפעיל את סעיף (ב) ולהסיק שהוא עפ"מ ביחס ל- $w$ , כנדרש.

⇒ אם ישנו עפ"מ שכל קשתותיו ב- $A$  ביחס ל- $w$ , אז נבחין ש- $f$  מונוטונית עולה חלש, ולכן ישנו עפ"מ כזה גם ביחס ל- $w \circ f$ . נבחין שקרוסקל בהכרח יחזיר אותו, כי הגדלנו את משקל כל הקשתות שאינן ב- $A$  ב- $\varepsilon > 0$  כלשהו, ולכן במידה וקרוסקל יתקל בשתי קשתות שבעבר ב- $w$  היו באותו המשקל, לאחר הרכבה ב- $f \circ w$  יינתן יתרון לאחת שבאה מהקבוצה  $A$  (ניסוח פורמלי של הטענה הזה ראינו בתרגול בשאלה בה ניסינו למצוא אלגוריתם למציאת עפ"מ תוך תעדוף לקשתות אדומות. בתרגול גם נתנו חסם עליון ל- $\varepsilon$  שבעברו קרוסקל עדיין עובד, אך אין צורך בזה כאן כי  $f$  מונוטונית עולה). סה"כ במקרה זה קרוסקל יחזיר את העפ"מ ולאחר שנוודא שכל קשתותיו ב- $A$  נחזיר תשובה חיובית, כנדרש.

• **סיבוכיות:** עלות הפעלת  $f$  תהיה  $\mathcal{O}(|E|)$ . עלות המיון תהיה  $\mathcal{O}(|A| + |V|)$ , ועלות הרצת קרוסקל תהיה  $\mathcal{O}(|E| \alpha(|V|))$  (ראינו בהרצאה שזה העלות של קרוסקל לאחר המיון). נסכם, ונחיל את העובדה ש- $|A| = \mathcal{O}(1)$ ,  $|E| \propto |V|$ :

$$\mathcal{O}(|E| + |A| + |V| + |E| \alpha(|V|)) = \mathcal{O}(k|V| + 1 + |V| + k|V| \alpha(|V|)) = \mathcal{O}(|V| \alpha(|V|))$$

כנדרש.

המשך בעמוד הבא

שני שחקנים נבונים  $A, B$  בוחרים מספרים מבין השורה, כאשר ניתן לבחור בכל פעם רק את המספר בקצה הקיצוני ביותר. לבסוף סכום המשחק מוגדר להיות סכום המספרים ש- $A$  בחר פחות סכום המספרים ש- $B$  בחר, כאשר  $A$  רוצה למקסם את ערך המספר בעוד  $B$  מנסה למנס את ערך המספר.  $A$  מתחיל. נמצא אלגוריתם יעיל המוצא את ערך המשחק בהינתן סדרת מספרים.

1. **הגדרת בעיות קטנות יותר:** בהינתן סדרה של מספרים  $\alpha_1 \dots \alpha_n$ , נגדיר את  $A[a, b]$  (כאשר  $0 \leq a < b \leq n$ ) להיות הפתרון לבעיה בעבור סדרת המספרים  $\alpha_a \dots \alpha_b$  כאשר  $A$  מתחיל, ואת  $B[a, b]$  להיות הפתרון לבעיה כאשר  $B$  מתחיל.

2. **הקשר לבעיה הגדולה יותר:** הקשר לבעיה הגדולה יותר: נחשב את  $[0, n]$  (כלומר  $a = 0, b = n$ ).

3. **הקשר של תת-בעיה לתת-בעיות קטנות יותר:** הקשר בין תתי בעיות לתת-בעיה גדולה יותר: כדי לחשב את  $C[a, b]$  כאשר  $C \in \{A, B\}$ , נפרק לפי המקרים הבאים:

• אם  $C = A$ , שחקן  $A$  בוחר את הקלף הימני  $\alpha_a$ , ונוסיף את  $[a + 1, b]$ . סה"כ ערך מקרה זה יהיה:

$$P_1 = \alpha_a + [a + 2, b]$$

• אם  $C = A$ , שחקן  $A$  בוחר את הקלף השמאלי  $\alpha_b$ , ונוסיף את  $[a, b - 1]$ . סה"כ ערך מקרה זה יהיה

$$P_2 = \alpha_b + [a + 2, b]$$

• אם  $C = B$ , שחקן  $B$  בוחר את הקלף הימני  $\alpha_a$ , ונוסיף את  $[a + 1, b]$ . סה"כ ערך מקרה זה יהיה

$$P_3 = -\alpha_a + [a + 2, b]$$

• אם  $C = B$ , שחקן  $B$  בוחר את הקלף השמאלי  $\alpha_b$ , ונוסיף את  $[a, b - 1]$ . סה"כ ערך מקרה זה יהיה

$$P_4 = -\alpha_b + [a + 2, b]$$

סה"כ, נגדיר:

$$A[a, b] = \max\{P_1, P_2\} \quad B[a, b] = \min\{P_3, P_4\}$$

במקרה הבסיס  $C[x, x + 1]$  נבחר את המספר היחיד שנשאר, ונוסיף לו סימן שלילי אם  $C = B$ .

4. **אופן החישוב:** נצטרך לממש את האלגוריתם רקורסיבי באמצעות ממואיזציה, כי אנו עלולים להתקל באותו מקרה הבסיס מספר רב של פעמים. נתחיל מחישוב של  $[0, n]$  ונמשיך רקורסיבית לפי המתואר לעיל.

5. **סיבוכיות:** נבחין שהסיבוכיות היא  $O(n^2)$  (גם מקום וגם זמן). זאת כי  $a, b \in \{1, \dots, n\}$  ו- $C \in \{1, 2\}$ , ולכן גודל הטבלה חסום ע"י  $|\{1, 2\}| \cdot |\{1, \dots, n\}|^2 = 2n^2 = O(n^2)$  (למעשה כמות הזכרון שנצטרך בפועל היא בדיוק  $n^2$  כי המקרים של  $A$  לא יתנגשו עם המקרים של  $B$ , אבל זה לא רלוונטי לניתוח אסימפטוטי), והעלות של כל קריאה רקורסיבית היא קבועה.

## המשך בעמוד הבא

## (5)

נתון גרף מכוון ואציקלי  $G = (V, E)$  ו- $s \in V$ . נגדיר משחק בו אריאנה (נסמנה  $A$ ) וביונסה (נסמנה  $B$ ) בונות יחדיו מסלול המתחיל מ- $s$ , שתיהן שחקניות רציונליות המשחקות לסירוגין כאשר  $A$  מתחילה. שחקנית  $C \in \{A, B\}$  מפסידה אם היא אינה יכולה להמשיך את מסלולה. בעבור  $C \in \{A, B\}$  שחקנית כללית, נסמן ב- $\bar{C}$  את השחקנית השנייה.

1. **הגדרת בעיות קטנות יותר:** נגדיר את  $c[v] \in \{F, T\}$  להיות התשובה לשאלה "האם במקרה זה  $C$  מנצחת במידה ו- $\bar{C}$  מתחילה בתור הבא.

2. **הקשר לבעיה הגדולה יותר:** נצטרך להחזיר את  $A[s]$ .

3. **הקשר של תת-בעיה לתת-בעיות קטנות יותר:** ננסה לפתור את  $c[v]$ . נסמן ב- $\text{adj}[v]$  את קבוצת השכנים של  $v \in V$ . אם  $\text{adj}[v] \neq \emptyset$ , נחזיר את:

$$c[v] = \bigvee_{u \in \text{adj}[v]} \neg \bar{c}[u]$$

כאשר  $\bigvee$  הוא logical or (בשפות תכנות, any) ו- $\neg$  הוא שלילה לוגית (אלו סימונים מבדידה 1). נוכיח שזה אכן עובד:

• אם  $c[v] = T$ : אזי ישנו קודקוד  $u$  שכן של  $v$  (כלומר  $u \in \text{adj}[v]$ ) כך שאם  $C$  תלך ל- $u$  ותאריך את המסלול, אז בהכרח  $\bar{C}$  תפסיד (אחרת  $\bar{C}$  תבחר מסלול שיגרום לה לנצח, ואז  $C$  מפסידה וזו סתירה).

• אם  $c[v] = F$ , אז לא משנה לאיזה קודקוד  $u \in \text{adj}[v]$  ממשיכה, בהכרח  $v$  תנצח, כלומר  $\bar{c}[u] = T$ .  $\forall u \in \text{adj}[v]$ : מכאן ש- $\neg \bar{c}[u] = F$ .

מקרה הבסיס של  $c[v]$  יהיה  $F$  אם  $\text{adj}[v] = \emptyset$ . הערה: השתמשו בהיות הגרף אציקלי כאשר הנחנו שיחנו תמיד מקרה בסיס שאפשר להגיע אליו. אחרת, ייתכן והיינו מסתובבים במעגל לנצח, והאלגוריתם לא היה נגמר.

4. **אופן החישוב:** נחשב באמצעות רקורסיה, כאשר נעשה ממואיזציה לשתי טבלאות (אחת ל- $A$  ואחת ל- $B$ ) בגודל  $|V|$  (לכל ערך  $v \in V$ ). סה"כ  $2|V| = \mathcal{O}(|V|)$  של זכרון.

5. **סיבוכיות:** אומנם יהיו רק  $\mathcal{O}(|V|)$  קריאות רקורסיביות, אך העלות של כל קריאה לחישוב  $c[v]$  היא  $d_{\text{out}}(v)$ . אך בגלל שעל כל  $v \in V$  נעבור פעם אחת רק פעמיים (אחרת נשתמש בטבלת הממואיזציה), סך העלות של הפעולות שיתבצעו בתוך כל קריאה יהיה:

$$\sum_{v \in V} 2 \cdot d_{\text{out}}(v) = 2 \sum_{v \in V} d_{\text{out}}(v) = 2|E| = \mathcal{O}(|E|)$$

מלמת לחיצות הידיים. סה"כ הסיבוכיות תהיה  $\mathcal{O}(|V| + |E|)$ . עלות המקום תהיה  $\mathcal{O}(|V|)$ , כגודל טבלת (או במקרה הזה, וקטור השורה) הממואיזציה.

**המשך בעמוד הבא**

## (6)

תנונה סדרת מספרים שלמים  $a_1 \dots a_n \in \mathbb{Z}$ . תת-סדרה רצופה מוגדרת היא ת"ס מהצורה  $a_i \dots a_{i+k}$ .

(א) נמצא אלגוריתם יעיל המוצא ת"ס רצופה במשקל מקסימלי. לשם כך נפעיל אלגוריתם שמשמש באלגוריתם אחר המשתמש בתכנון דינמי.

1. הגדרת בעיות קטנות יותר: נגדיר ב- $[i]$  את המשקל המקסימלי של ת"ס רצופה המסתיימת ב- $i$ .

2. הקשר לבעיה הגדולה יותר: נוכל לחשב את  $[i] := \max_{i \in \{1 \dots n\}} [i]$  כדי למצוא את התא שבו הסדרה נגמרת, ואז לעבור בזמן לינארי את כל על כל  $1 \leq \ell \leq m$  האפשריים ומציאת ה- $\ell$  המקסימלי כך שהסכום  $a_\ell + \dots + a_m$  מקסימלי, כלומר להגדיר את אינדקס ההתחלה להיות:

$$\max_{\ell \in \{1 \dots m\}} \sum_{i=\ell}^m a_i$$

ואז נחזיר את  $a_\ell \dots a_m$ .

3. הקשר של תת-בעיה לתת-בעיות קטנות יותר: נבחין שמתקיים הקשר:

$$[i] = a_i + \max\{0, [i-1]\}$$

נוכיח זאת באמצעות פירוק למקרים. נאמר ש- $a_\ell \dots a_i$  הסדרה המקסימלית המסתיימת ב- $i$ .

• אם  $\ell = i$ , אז הסדרה המקסימלית הינה פשוט  $a_i$ . דהיינו צירוף של  $a_i + a_{i-1} + \dots$  סדרה המסתיימת ב- $a_{i-1}$  רק יקטין את משקל הסדרה. במקרה זה  $[i-1] < 0$  כלומר נאמר ש- $[i] = a_i$  שזהו אכן משקל הסדרה  $a_i$ .

• אם  $\ell \neq i$ , אז משום שהסדרה מסתיימת ב- $i$ , בהכרח  $\ell < i$ . מכאן שהסדרה המקסימלית המסתיימת ב- $i$  מהצורה  $a_\ell + \dots + a_{i-1} + a_i$  (ייתכן  $\ell = i-1$ ). במקרה זה יהיה הכי אופטימלי להוסיף את ערך הסדרה המקסימלית שמסתיימת ב- $i-1$  לסדרה שלנו, ובהכרח מתקיים  $[i-1] \geq 0$  (אחרת יהיה עדיף שהסדרה תהיה סינגלטון  $a_i$ , וזו סתירה לכך ש- $\ell \neq i$ ). סה"כ במקרה זה אכן  $[i] = a_i + [i-1] = a_i + \max\{0, [i-1]\}$  כנדרש.

כל זאת בעבור מקרה בסיס  $a[0] = 0$  (החלופה לסדרה ריקה). לפעמים סדרה ריקה תהיה עדיפה על פני סינגלטון או סדרה אחרת, לדוגמה במקרה שבו כל המספרים שליליים).

4. אופן החישוב: נחשב איטרטיבית, מ- $[1]$  עד  $[n]$ . נדרוש זכרון של  $O(1)$  שכן רק נצטרך באיטרציה ה- $i$  לשמור את הערך הקודם של  $i-1$ , ואת הערך הגבוה ביותר שמצאנו עד עכשיו. את  $m := \max_{i \in \{1 \dots n\}}$  נוכל לחשב "on the fly" באמצעות כך שכל פעם שאנו נתקלים בערך  $i$  שעבורו  $[i]$  גבוהה יותר, נעדכן את הערך המקסימלי שמצאנו. לבסוף נעבור על כל ה- $\ell \in \{1 \dots m\}$  המתאימים כדי למצוא מיקום התחלה.

5. סיבוכיות: הלולאה תדרוש  $O(n)$  איטרציות כדי למצוא אינדקס סיום, ו- $O(n)$  בלולאה נוספת (במקרה הגרוע) איטרציות כדי למצוא אינדקס התחלה. הכל מתבצע בזמן קבוע. סה"כ  $O(n)$ . סיבוכיות המקום קבועה.

(ב) נמצא אלגוריתם שמוצא שתי ת"ס רצופות זרות שסך המשקלים שלהן מקסימלי.

**שאלה זו לא נפתרה באמצעות תכנון דינמי.** אפשר בקלות להמיר את הפתרון לפתרון בתכנון דינמי, אבל זה סתם יוצא יותר מסורבל, ופתרון באמצעות תכנון דינמי אינה אחת מהדרישות של השאלה.

• **אלגוריתם:** נסמן ב- $[i]$  את אינדקס הסיום של תת-הסדרה הרצופה המקסימלית ב- $1 \dots i$ , וב- $[i]$  את אינדקס ההתחלה של תת-הסדרה הרצופה המקסימלית ב- $i \dots n$ , וכן את משקל הסדרות האלו. נבחין שהרצת האלגוריתם מהסעיף הקודם (ללא שלב הסיום שלו) מחזירה את הסדרה הארוכה ביותר המסתיימת לכל היותר באינדקס ה- $i$ , ואת זה נוכל להכניס ישירות ל- $[i]$  (זאת כי במהלך הריצה הוא שומר לעצמו את ה- $j \in \{1 \dots i\}$  כך שסדרה המסתיימת ב- $j$  היא המקסימלית מבין כל הסדרות שראינו עד עכשיו). נוכל לעשות דבר דומה כדי להתחל את, ע"י כך שנפעיל את האלגוריתם מהסעיף הקודם עבור  $a_n, a_{n-1} \dots a_1$  במקום  $a_1 \dots a_n$ . סה"כ, נחשב את ה- $i$  שממקס את הביטוי  $[i] + [i+1]$ . ב- $i$  שמצאנו יהיה שמור אינדקס הסיום של הסדרה הראשונה, נסמנו  $m_1$ , ואינדקס ההתחלה של הסדרה השנייה, נסמנו  $\ell_2$ . בדומה לסעיף קודם, נוכל לחשב  $\ell_1 \in \{1 \dots m_1\}$  ו- $m_2 \in \{\ell_1 \dots n\}$  אינדקס ההתחלה של הראשונה ואינדקס הסיום של השנייה בהתאמה, ע"י מעבר לינארי (ראשית על כל ה- $\ell_1$  האפשריים, ושנית על כל ה- $m_2$  האפשריים) ומציאת הערכים שממקסמים את הסדרות. לבסוף נחזיר את  $a_{\ell_1} \dots a_{m_1}$  ואת  $a_{\ell_2} \dots a_{m_2}$ .

• **נכונות:** נכונות הערכים של, מובטחת מנכונות סעיף קודם. משום שעל הסדרות להיות זרות, בהכרח ישנו אינדקס  $i$  כלשהו שלכל ביותר הסדרה הראשונה נגמרת, והסדרה השנייה מתחילה איפשהו אחריו (פוטנציאלית, מיד באינדקס שאחריו. גם אם היא ריקה). מהגדרת, ערך זה יהיה  $[i] + [i+1]$ . עכשיו מנכונות הערכים שב-, סיימנו.

• **סיבוכיות:** הרצת האלגוריתם מהסעיף הקודם פעמיים תארוך  $2O(n) = O(n)$ . חישוב ה- $i$  המקסימלי יארוך  $O(n)$  (מעבר על כל האפשרויות של הסכום  $[i] + [i+1]$ ), ולבסוף מציאת  $\ell_1$  תארוך  $O(n)$  ומציאת  $m_2$  תארוך גם היא  $O(n)$ . סה"כ  $O(n)$ .

# (7)

נתונות פונקציות  $f_1 \dots f_m: \{1 \dots m\} \rightarrow \mathbb{Z}$  ומספר טבעי  $n \in \mathbb{N}$ . נמצא אלגוריתם יעיל שמוצא מספרים שלמים  $x_1 \dots x_m$  שממקסמים את הסכום  $\sum_{i=1}^m f_i(x_i)$  כאשר  $\sum_{i=1}^n \leq n$ .

1. הגדרת בעיות קטנות יותר: נסמן ב- $[n', m']$  את פתרון הבעיה (כלומר את וקטור המספרים) בעבור בחירת  $m'$  משתנים  $x_1 \dots x_{m'} \in \{0 \dots x_m\}$  (שימו לב שגם בעבור  $m' < m$  נוכל לבחור  $x_i = m$ ) תחת האילוץ ש- $\sum x_i < n'$ .

2. הקשר לבעיה הגדולה יותר: נחזיר את  $[n, m]$  בעבור  $n, m$  המקוריים.

3. הקשר של תת-בעיה לתת-בעיות קטנות יותר: ישנו מקרה בסיס ראשון בו  $m = 0$  ונחזיר 0 מספרים, ומקרה בסיס בו  $n = 0$  ואז נחזיר  $m$  פעמים את  $x_i = 0$ .

נסמן ב- $(x_1 \dots x_m)$  המשקל של הפתרון  $[n', m']$ . בעבור  $n', m' \in \mathbb{N}$  כלליים, נבחין שנוכל לחשב את:

$$\max_{\ell \in \{1 \dots m\}} ([n' - \ell, m' - 1] + f(\ell))$$

בעבור ה- $\ell$  הממקסמים את הביטוי לעיל, נחזיר את השרשור של  $[n' - \ell, m' - 1]$  עם  $x_m = \ell$ .

4. אופן החישוב: רקורסיבי עם טבלת ממואיזציה.

5. סיבוכיות: הטבלה מגודל  $m \cdot n$ , וכל חישוב יערך  $m$ , סה"כ  $O(m^2 n)$  עם זכרון של  $O(nm)$ .

שחר פרץ, 2026

קומפל ב-L<sup>A</sup>T<sub>E</sub>X ויוצר באמצעות תוכנה חופשית בלבד